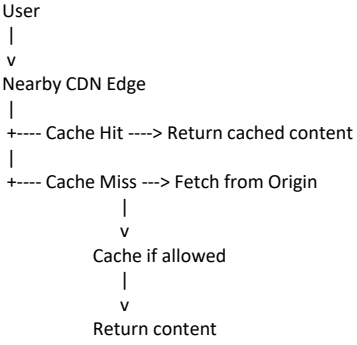
Content Delivery Network (CDN)

A Content Delivery Network (CDN) is a distributed network of servers that caches content closer to users. It helps reduce latency, decrease the load on the origin server, and deliver content faster to users around the world.
Suppose an application runs in a data center in Virginia. A user in New York requests an image, and the request travels a relatively short network path. However, when a user in India requests the same image, the request has to travel a much longer network path to reach the origin server, increasing latency.
When millions of users around the world request the same static files, the origin server repeatedly sends identical content. This wastes bandwidth and server resources.
A CDN solves this problem by storing copies of popular content at edge servers in different geographic locations. Users can then retrieve content from a nearby edge instead of always contacting the origin server.
A **cache hit** occurs when the requested content is already available at the CDN edge, so it can be served quickly.
A **cache miss** occurs when the content is not available at the edge. The CDN fetches it from the origin and caches a copy if caching is allowed.



1. Edge Locations

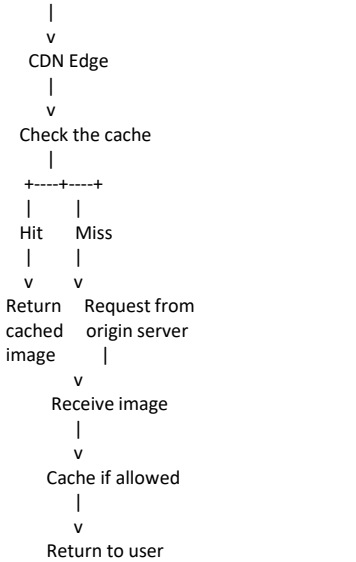
CDN servers are deployed in geographic locations close to users. These locations are commonly called **edge locations** or **Points of Presence (PoPs)**.
Each edge location contains servers that accept user requests, store cached responses, and forward requests to the origin server when the required content is not available locally.
For example, an application may have its origin server in the United States while its CDN has edge locations in Singapore, Mumbai, London, Tokyo, and other cities.
When a user in India requests an image, the CDN can serve it from a nearby edge location rather than sending the request all the way to the United States.



The shorter network path usually reduces latency. It also reduces the work performed by the origin server because many requests are served directly by CDN edges.

2. How a CDN Request Works

Suppose a user requests an image named logo.png. The request is routed to a nearby CDN edge server, and the edge checks whether the image is already cached.
If the image is available in the cache, the request is a cache hit. The CDN returns the image without contacting the origin server.
If the image is not available, the request is a cache miss. The CDN requests the image from the origin, receives it, stores a copy if allowed, and returns it to the user.
When another user in the same region requests the same cached image, the CDN can serve it directly from the edge. This is caching at a global scale.



The Origin

The origin is the authoritative source from which the CDN retrieves content. It can be an application server, an object storage service, or another backend system such as a load balancer or API gateway.
For example, a website may store images and videos in object storage, with a CDN placed in front of that storage.
Users request content through the CDN. The CDN contacts the origin when the required object is missing or when its cached copy needs to be refreshed.

This prevents the origin from having to serve every image, JavaScript file, or video segment directly. The origin mainly handles cache misses, dynamic requests, writes, and content that should not be cached.

3. What CDNs Cache

CDNs are commonly used for static content, including images, JavaScript files, CSS files, fonts, videos, software downloads, and other files that do not change with every request.
These files are good candidates for caching because the same content may be requested millions of times by different users.
Modern CDNs can also cache some dynamic content, such as product pages or API responses, if the same response can safely be reused across users.
The important question is whether different requests can share the same cached response.
Content is a good candidate for CDN caching when it is read frequently, does not change on every request, can be shared by many users, and can tolerate being slightly stale for a known amount of time.
Account pages, shopping carts, payment flows, admin pages, and responses that depend on private cookies or login state are generally poor candidates for shared CDN caching.

4. HTTP Caching Headers

The origin server usually tells the CDN how content should be cached by using HTTP caching headers. One of the most important headers is Cache-Control.
For example, the origin may specify that an image can be cached for one hour. During that time, the CDN edge can serve the cached copy without contacting the origin. Once the cache lifetime expires, the CDN may need to validate the cached copy or fetch the content again.
These headers allow the application to control how long content remains fresh. Static assets may be cached for a long time, while frequently changing data may need a much shorter cache lifetime.
The important Cache-Control directives are:

- public: Shared caches, including CDNs, may store the response.
- private: The response is intended for a single user and should not be stored in a shared cache.
- max-age: Specifies how many seconds a cache can treat the response as fresh.
- s-maxage: Specifies the freshness lifetime for shared caches, such as CDNs.
- no-store: Tells caches not to store the response.
- immutable: Indicates that the content will not change while it remains fresh.
- stale-while-revalidate: Allows a stale response to be served briefly while the cache refreshes it in the background.

ETag and Last-Modified

ETag and Last-Modified help the CDN validate whether its cached content has changed.
When a cached copy expires, the CDN can ask the origin whether the object has changed. If the object has not changed, the origin can return a 304 Not Modified response instead of sending the complete object again.
This reduces unnecessary data transfer between the origin and the CDN.

5. TTL (Time to Live)

TTL defines how long a CDN can reuse a cached object before considering it expired.
A long TTL improves performance and reduces requests reaching the origin, but it increases the possibility of serving outdated content.
A short TTL keeps content fresher, but more requests reach the origin because cached objects expire sooner.
The correct TTL depends on how frequently the content changes and how much staleness users can tolerate.
For example, versioned JavaScript files can have a very long TTL because each version has a different URL. Frequently changing API responses may need a TTL of only a few seconds or minutes.
The main trade-off is:

```
Long TTL
  |
  +--> Better cache reuse
  +--> Lower origin traffic
  +--> Greater risk of stale content
Short TTL
  |
  +--> Fresher content
  +--> More frequent origin requests
  +--> Potentially lower cache hit ratio
```

6. Purging and Versioned URLs

Sometimes a system cannot wait for a cached object to expire naturally. For example, suppose an incorrect image is published and needs to be replaced immediately. The old image may already be cached at CDN edge locations around the world.
Two common approaches are purging cached content and using versioned URLs.

Purging

Purging, also called cache invalidation, removes or stops serving a cached object so that the CDN can retrieve the updated version from the origin.
Many CDN providers allow a system to purge a single URL, an entire path prefix, or a group of objects associated with specific tags.
However, global invalidation can take time, and providers may limit the number of purges allowed.
Another concern is that every purged object may become a cache miss. The CDN then has to fetch those objects from the origin again. A large purge can therefore create a sudden spike in origin traffic.

Versioned URLs

Instead of replacing a file while keeping the same URL, a system can publish the updated file using a new URL.
For example, instead of replacing logo.png, the application can publish logo.v2.png.
The new URL creates a new cache entry, allowing the CDN to treat the updated file as a separate object.
Versioned URLs are especially useful for static assets. Suppose the application uses app.js. If the file changes but the URL stays the same, some users may continue receiving the older cached version.
Instead, the build system can generate a filename such as app.a7b9.js. When the content changes, the hash changes, and the new deployment references a different URL.
The CDN can cache each version for a long time because the content associated with that URL does not change.
This provides both strong caching and reliable updates without requiring every edge to immediately replace the previous copy.
For content that must retain the same URL, such as an HTML page, the application can use a short TTL, purge the cached content when it changes, or rely on ETag validation.

7. How CDN Routing Works

A CDN is useful only when users are routed to a nearby, healthy edge location that can serve their requests efficiently.
There are two common routing approaches: DNS-based routing and Anycast.

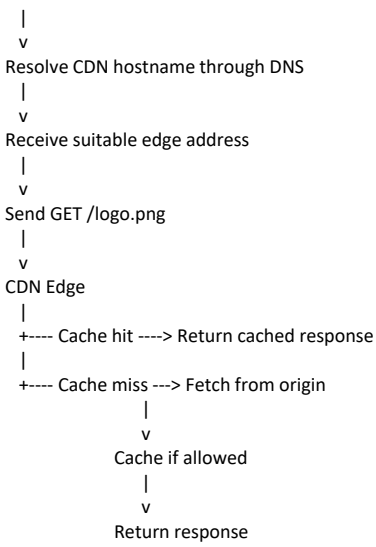
DNS-Based Routing

When a user looks up the CDN hostname, the CDN's DNS system returns an IP address associated with a suitable edge location.
For example, a user in India may receive an address associated with a nearby CDN edge.

Anycast

With Anycast, multiple edge locations advertise the same IP address. Internet routing directs the request toward a nearby available location according to network routing decisions.
The exact routing mechanism depends on the CDN provider.
The geographically closest edge is not always the one selected. Network speed, edge capacity, and server health also affect the decision.

The overall goal is to direct the user to an edge that can serve the request efficiently.



8. Pull CDNs and Push CDNs

There are two common ways content can reach CDN edge locations: pull-based caching and push-based distribution.

Pull CDN

With a pull CDN, content is loaded into an edge cache when someone requests it.
For example, suppose an image has never been requested from the Tokyo edge. The first request results in a cache miss, so the Tokyo edge fetches the image from the origin and stores a copy.
Later requests for the same object can be served directly from that edge, assuming the cached copy is still valid.
This approach is simple because the application does not need to upload every file to every edge location ahead of time.
The main disadvantage is that the first request in a region may have higher latency because the content must be fetched from the origin.

Push CDN

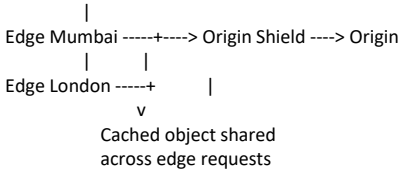
With a push CDN, the application uploads content to the CDN before users request it. The CDN then distributes that content across its network.
This is useful when the application knows which files need to be available in advance. For example, large software downloads or media files can be distributed before a planned launch.
Push CDNs can avoid the initial cache miss when the content has already been distributed to the relevant edge locations.
However, they require more coordination because the application must manage what content is uploaded and when it changes.

Aspect	Pull CDN	Push CDN
When content reaches the edge	When first requested	Before users request it
First request in a region	May be a cache miss with higher latency	Can already be cached
Content management	CDN loads content automatically on demand	Application explicitly manages uploads
Good fit	General web content	Large, planned releases

In practice, pull-based caching is more common for general web content.

9. Origin Shield

Large CDN deployments sometimes add an additional cache layer between edge servers and the origin. This layer is called an **origin shield**.
Suppose 100 edge locations all miss the same popular object at roughly the same time.
Without an origin shield, all 100 edge locations might send separate requests to the origin for the same object.
With an origin shield, the edge locations request the object from an intermediate cache. The shield may fetch the object from the origin only once and then satisfy the requests from the cached copy.



Origin shielding reduces duplicate requests to the origin. It is particularly useful when many edge locations need the same content at approximately the same time.

10. Cache Stampedes

A CDN can experience a cache stampede just like an application-level cache.
Suppose a popular object expires at many edge locations around the same time. Those edges may all experience cache misses and request the same object from the origin.
This can generate a sudden spike in origin traffic, potentially affecting performance or overwhelming the backend.
Several techniques help reduce cache stampedes.
TTL jitter: A small random variation is added to expiration times so that many cached copies do not expire simultaneously.
Stale-while-revalidate: The CDN continues serving a slightly stale object while refreshing it in the background.
Request coalescing: A single origin fetch is used to satisfy multiple simultaneous cache misses for the same object, rather than sending a separate origin request for every miss.
Origin shielding: Misses from multiple edge locations are funnelled through an intermediate cache, reducing duplicate requests reaching the origin.
For example, stale-while-revalidate allows an edge to serve the existing copy while one background request fetches the latest version. This reduces the chance of a sudden burst of requests reaching the origin.

11. Personalized Content

CDNs work best when many users can safely share the same cached response. Personalized content makes this more difficult.
Suppose an API response contains the current user's name and account information. The CDN cannot safely serve that same response to another user because the data belongs to the original user.
One option is to include user-specific information in the cache key. However, this creates separate cache entries for different users and reduces the benefits of sharing cached content.
For this reason, systems often separate shared content from personalized content.

The CDN serves common assets and public data, while personalized requests continue to the application backend.
Private user data should not be stored in a shared CDN cache unless the design has been carefully reviewed. An incorrect caching rule can expose one user's information to another user.

12. Cache Keys

A CDN cache key determines whether two requests can use the same cached response.
The URL is usually part of the cache key, but query parameters, headers, cookies, and other request attributes may also be included.
Correct cache-key design is important because an incorrectly configured key can reduce the cache hit ratio or cause the wrong content to be returned.

Cache Key That Is Too Narrow

Suppose two requests return exactly the same image, but they contain different query parameters that do not affect the image.
If the CDN includes every query parameter in the cache key, it may store two copies of the same object.
This creates unnecessary cache entries and reduces the hit ratio.
Including every cookie in the key can cause a similar problem because almost every request may appear unique.

Cache Key That Is Too Broad

Sometimes a query parameter or request header genuinely changes the response.
For example, if the requested image width changes the returned image, the width must be considered when determining whether the cached response can be reused.
Similarly, if a language header changes the response, ignoring it could cause the CDN to return English content to a user who requested Spanish.
A cache key that ignores important request attributes can therefore return incorrect content.
Most CDN providers allow the application to choose which query parameters, headers, and cookies are included in the cache key.
For public content, keep the key small to maximize reuse. Include cookies or other request attributes only when the response genuinely depends on them.

13. Caching API Responses

CDNs are not limited to images, CSS, and JavaScript. Some API responses can also be cached at the edge.
For example, suppose a public API returns the current list of supported countries. Every user receives the same response, and the information changes infrequently.
That response is a good candidate for CDN caching because the same data can be reused by many users around the world.
Requests can then be answered from nearby CDN edges without reaching the application servers. This reduces latency and backend load.
However, APIs require careful cache rules.
If a response depends on authentication, cookies, language, location, or other request attributes, the cache key and privacy rules must account for those dependencies.
Endpoints such as /api/me and /checkout should generally bypass shared caching because their responses may contain private or user-specific information.

14. Large Files and Range Requests

CDNs are especially useful for delivering large files such as videos, software packages, and downloads.
Serving a one-gigabyte file repeatedly from a single origin can consume substantial bandwidth. With a CDN, popular files can be served by edge infrastructure rather than repeatedly transferred from the origin.
CDNs can also support **HTTP range requests**, which allow a client to request only a specific part of a file instead of downloading the entire object.
For example, a video player may request particular portions of a video as the user watches or skips to different points.
The CDN can serve the requested byte ranges without forcing the origin to handle every stream directly.

15. Reliability and Security

A CDN can improve application reliability as well as performance.
If the origin becomes temporarily unavailable, some CDNs can continue serving cached content. This can keep static parts of an application working even during backend problems.
The CDN also absorbs a large amount of normal traffic before it reaches the origin. This reduces bandwidth consumption and connection load on the origin server.
It is especially useful during product launches, sales, and other events that cause sudden traffic spikes.
Many CDN platforms also provide security features such as denial-of-service attack protection, Web Application Firewall (WAF) rules, bot filtering, rate limiting, and TLS management.
These features help reduce unwanted traffic reaching the origin, but they do not replace application-level security.
A CDN is not a replacement for a reliable backend. Content that is not cached still needs to be served by the origin, and backend availability remains important.

16. CDN Metrics

Several metrics help determine whether a CDN is working effectively.
Cache hit ratio: This measures the proportion of requests served from the cache. A high hit ratio generally means more traffic is served from edge locations instead of reaching the origin.
Origin request volume: This measures how many requests reach the origin. If CDN traffic increases while origin traffic remains relatively flat, the CDN is absorbing the additional load.
Latency: This measures how long requests take to receive a response. Users should generally experience lower response times when content is served from a nearby edge.
Other useful metrics include bandwidth usage, cache evictions, error rates, and the amount of traffic served using stale content.
A high overall hit ratio can hide problems with specific endpoints. Therefore, check the hit ratio for the routes that matter most to users and to origin infrastructure costs.

Response Headers for Debugging

Response headers can help identify what happened when a CDN handled a request.
Age indicates how many seconds the cached response has been stored or how old the response is in the cache, according to the header's semantics.
A header such as X-Cache may indicate whether the request was a cache hit or a miss.
The exact header names and their behavior vary by CDN provider.

17. CDN Providers

Major CDN providers include Akamai, Cloudflare, Fastly, Amazon CloudFront, Google Cloud CDN and Media CDN, and Azure Front Door.
The right provider depends on the needs of the system.
Important comparison factors include where the users are located, compatibility with the existing cloud infrastructure, flexibility of cache rules, purge speed, support for origin shielding and request coalescing, logging and metrics, security features, and pricing.
CDN pricing often includes bandwidth usage, request counts, purges, and additional features. Video delivery and large downloads can become expensive depending on the traffic pattern.
For example, a team already running its application on AWS may prefer Amazon CloudFront because it integrates naturally with the rest of its infrastructure. A media company may prioritize a provider's video delivery capabilities.